Plume Network

Security Review

Cantina Managed review by:
R0bert, Lead Security Researcher
Slowfi, Security Researcher

January 7, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 High Risk . . . 4
3.1.1 Instant redeem checks wrong balance and reverts . . . 4
3.1.2 Management fee configured in `NestAccountant` is never applied . . . 4
3.1.3 NestAccountant unusable (no init/auth) . . . 5
3.2 Medium Risk . . . 6
3.2.1 Rate bounds unusable due to uint16 scaling . . . 6
3.2.2 Invalid exchange rate accrues double fees . . . 6
3.3 Low Risk . . . 6
3.3.1 Fee claims lose value on decimal downscale . . . 6
3.3.2 Fee accrual skipped when paused on invalid update . . . 7
3.3.3 `fulfillRedeem` can burn queued shares for zero assets . . . 7
3.3.4 `NestVaultOFT` lacks `share()` function required by EIP-7575 . . . 8
3.3.5 RedeemRequest event misnamed final field . . . 8
3.3.6 Deposit event omitted in `NestVaultCore._deposit` . . . 9
3.3.7 EIP-712 domain can drift on `authorizeOperator` . . . 9
3.4 Gas Optimization . . . 9
3.4.1 Cache asset token in redeem paths . . . 9
3.4.2 RateProvider lookup on base-asset fee claims . . . 10
3.4.3 Avoid double storage read in `getRateSafe` . . . 10
3.5 Informational . . . 10
3.5.1 Cross-chain shares minted without moving backing . . . 10
3.5.2 `updateRedeem` cannot increase pending shares . . . 11
3.5.3 Unused code . . . 12
3.5.4 Predicate approvals not bound to function/args . . . 12
3.5.5 `SupportsInterface` NatSpec mismatches implementation . . . 12
3.5.6 Initialize docstring misstates LayerZero delegate . . . 13
3.5.7 Auth gating conflicts with ERC7540 public entrypoints . . . 13
3.5.8 Avoid usage of magic numbers . . . 13

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Plume is a public, EVM-compatible blockchain built for the next evolution of Real World Assets (RWAs).

From Nov 24th to Nov 29th the Cantina team conducted a review of nest-contracts on commit hash cc5919fe. The team identified a total of **23** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	3	3	0
Medium Risk	2	2	0
Low Risk	7	7	0
Gas Optimizations	3	3	0
Informational	8	6	2
Total	**23**	**21**	**2**

2.1 Scope

The security review had the following components in scope for nest-contracts on commit hash cc5919fe:

```
contracts
├── NestAccountant.sol
├── NestShare.sol
├── NestVault.sol
└── NestVaultOFT.sol
```

3 Findings

3.1 High Risk

3.1.1 Instant redeem checks wrong balance and reverts

Severity: High Risk

Context: NestVaultCore.sol#L497-L501

Description: The instant redeem path measures the vault's own asset balance before/after `_exit`:

```
function instantRedeem(uint256 _shares, address _receiver, address _owner)
    public
    requiresAuth
    nonReentrant
    returns (uint256 _postFeeAmount, uint256 _feeAmount)
{
    SafeTransferLib.safeTransferFrom(ERC20(SHARE), _owner, address(this), _shares);

    (_postFeeAmount, _feeAmount) = _convertToAssetsForInstantRedeem(_shares);

    uint256 _initialBalance = ERC20(asset()).balanceOf(address(this));

    _exit(_receiver, ERC20(asset()), _postFeeAmount, address(this), _shares);

    uint256 _finalBalance = ERC20(asset()).balanceOf(address(this));

    uint256 _actualTransfer = _finalBalance - _initialBalance;

    if (_actualTransfer < _postFeeAmount) {
        revert Errors.TRANSFER_INSUFFICIENT();
    }
}
```

But `_exit` forwards assets straight to the `receiver`:

```
function _exit(address _to, ERC20 _asset, uint256 _assets, address _from, uint256
↪   _shares) internal override {
    BoringVault(SHARE).exit(_to, _asset, _assets, _from, _shares);
}
```

When `_receiver` is not `address(this)`, the vault's balance does not change, `_actualTransfer` stays zero and the `TRANSFER_INSUFFICIENT` check always reverts. Instant redeem therefore fails for any external recipient.

Recommendation: Consider measuring the receiver's balance delta instead of the `NestVaultCore`'s.

Plume Network: Fixed in commit 6cef45e7.

Cantina Managed: Fix verified.

3.1.2 Management fee configured in `NestAccountant` is never applied

Severity: High Risk

Context: NestAccountant.sol#L358, NestVaultCore.sol#L869-L871, NestVault.sol#L103-L105

Description: The function `updateExchangeRate` from contract `NestAccountant` computes management fees using `getManagementFee()`. In the concrete `NestVault` deployment, this call resolves to the override `getManagementFee` from contract `NestVault`, which returns the value from `NestVaultCore.getManagementFee()` instead of using the accountant state.

`NestVault.getManagementFee()` resolves to the `NestVaultCore` implementation, which reads `fees[Fees.Management]`:

```
// NestVault
function getManagementFee() public view override(NestVaultCore, NestAccountant) returns
↪  (uint256) {
    return NestVaultCore.getManagementFee();
}

// NestVaultCore
function getManagementFee() public view virtual returns (uint256) {
    return _getNestVaultCoreStorage().fees[DataTypes.Fees.Management];
}
```

At the same time, the effective management fee is stored in the accountant state (`NestAccountantStorage.accountantState.managementFee`) and can be configured via the NestAccountant initializer or `updateManagementFee`. However, because `updateExchangeRate` calls `getManagementFee()` and this call resolves to the `NestVaultCore` implementation, the configured accountant fee is ignored. Management fees are therefore not accrued into `feesOwedInBase`, and external callers querying `getManagementFee()` on `NestVault` will always observe a zero rate even when a non-zero management fee was configured in the accountant.

Recommendation: Consider to make `NestVault.getManagementFee` return the management fee stored in the accountant state (for example, by delegating to `NestAccountant.getManagementFee`) so that `updateExchangeRate` uses the configured fee. Alternatively, consider to remove or align the `Fees.Management` entry in `NestVaultCore` so there is a single source of truth for the management fee, and ensure both internal accrual logic and external getters use this source consistently.

Plume Network: Fixed in commit b43d6889.

Cantina Managed: Fix verified.

3.1.3 NestAccountant unusable (no init/auth)

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description:

- Introduced in commit b43d6889

`NestAccountant` inherits `AuthUpgradeable` but never invokes `__Auth_init` and exposes only internal initializers. The constructor calls `_disableInitializers()`, and there is no public/external initializer that sets owner or authority or populates accountant state. As deployed behind a proxy, `owner()` and `authority()` remain zero, making `requiresAuth` revert for every admin method such as `pause`, `unpause`, `updateExchangeRate`, bound updates, fee updates and rate provider configuration. The initial accountant state also starts at zeros, so the first `updateExchangeRate` with a nonzero rate passes through the `_invalid` branch, sets `isPaused = true` and exits without updating `exchangeRate`. Because auth is broken, `unpause` is unreachable.

Impact: any vault wired to this accountant will read a zero or paused rate, causing rate validation to revert and blocking deposits/redemptions until migrated to a different accountant.

Recommendation: Add a public/external initializer that calls `__Auth_init(owner, authority)` and the accountant initializer and seed a nonzero starting exchange rate and timestamp so the first update cannot auto-pause. For example:

```
function initialize(..., address _owner, Authority _authority) external initializer {
    __Auth_init(_owner, _authority);
    __NestAccountant_init(...);
}
```

Plume Network: Fixed in commit 5c80360d.

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Rate bounds unusable due to uint16 scaling

Severity: Medium Risk

Context: NestAccountant.sol#L259-L270

Description: The exchange-rate bounds are stored as uint16 but treated as 1e6-scaled limits. `updateUpper` insists the input be at least 1e6 (interpreted as 100% in the 1e6 scale), yet a uint16 argument can never exceed 65535, so the maximum upper bound that can be set is ~6.5%. `updateExchangeRate` then compares the new rate against `_currentExchangeRate.mulDivDown(allowedExchangeRateChangeUpper, 1e6)`, permitting at most ~6.5% increase from the previous rate. Any realistic update like, for example, attempting to keep the rate roughly the same, trips the guard and pauses the contract, disabling live rate updates and leaving fee accounting and pricing frozen.

Recommendation: Store the rate-change bounds in a wider type (uint32 or uint256) and keep them in the same 1e6 scale used by the guards. Update the setters to accept and store the full-range 1e6-scaled inputs and adjust initialization to use the wider type so bounds can be configured meaningfully without truncation.

Plume Network: Fixed in commit da85d43a.

Cantina Managed: Fix verified.

3.2.2 Invalid exchange rate accrues double fees

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description:

- Introduced in commit a169b30d

In `updateExchangeRate` the invalid path for too-soon or out-of-bounds submissions still accrues management fees and sets `isPaused`, but does not advance `lastUpdateTimestamp` or `totalSharesLastUpdate`. A call on the invalid path computes `_timeDelta` from the previous timestamp, converts `totalSharesLastUpdate` at the current exchange rate, and adds the resulting `_newFeesOwedInBase` to `feesOwedInBase`, then pauses. The next valid call reuses the same `lastUpdateTimestamp` to compute `_timeDelta` and charges fees again for the same elapsed interval, inflating accrued fees beyond the intended amount. Impact: any invalid update attempt double-charges the elapsed period, increasing what users must pay when fees are claimed.

Recommendation: When the update is invalid either skip fee accrual entirely or, if accrual is desired, also advance `lastUpdateTimestamp` (and `totalSharesLastUpdate` if appropriate) while pausing so the same interval is not billed again on the next valid update. Align the function comment with the chosen behavior.

Plume Network: Fixed in commit fa8ddd3f.

Cantina Managed: Fix verified.

3.3 Low Risk

3.3.1 Fee claims lose value on decimal downscale

Severity: Low Risk

Context: NestAccountant.sol#L205-L215

Description: Fee claims call `changeDecimals` to convert `feesOwedInBase` into the payout token's decimals. When scaling down, `changeDecimals` divides and floors, discarding any sub-unit remainder before the conversion by the fee-asset rate and then zeroing `feesOwedInBase`. The dropped remainder cannot be re-claimed and is permanently lost by the payout recipient. Example for 18-decimal base and 6-decimal payout: a 1.23456789 base-asset fee floors to 1.234567 after `changeDecimals`, losing 0.00000089 base units; with a $3000basetokenthatis$ 2.67 per claim, and with 0-decimal payout tokens nearly

a full token can be lost on each claim. Impact: protocol fees are systematically underpaid to the recipient, with loss per claim bounded by just under one unit of the payout token's precision but accumulating over many claims.

Recommendation: Use rounding-up or carry the discarded remainder forward. For example, when scaling down keep `remainder = _amount % 10 ** (_fromDecimals - _toDecimals)` and either add one unit if `remainder > 0` or accumulate the remainder back into `feesOwedInBase` so dust is paid out on the next claim. Similar rounding-up should be applied anywhere `changeDecimals` is used for payouts.

Plume Network: Fixed in commit fe87a31d.

Cantina Managed: Fix verified.

3.3.2 Fee accrual skipped when paused on invalid update

Severity: Low Risk

Context: NestAccountant.sol#L323-L369

Description: `updateExchangeRate` pauses on failed bound or delay checks but still overwrites `exchangeRate`, `totalSharesLastUpdate` and `lastUpdateTimestamp` and emits `ExchangeRateUpdated`. The pause branch skips management-fee accrual. An authorized updater can submit an out-of-bounds or too-soon rate to set `isPaused = true` and reset `lastUpdateTimestamp` to now while keeping the new rate, then unpause later. The elapsed period between the last valid update and the pause is never charged fees, creating an accounting hole while the rate change persists.

Recommendation: On bound/delay failures, revert or avoid updating rate/timestamps in the pause branch. Alternatively accrue fees up to the last valid timestamp before pausing or require a validated rate before unpausing.

Plume Network: Fixed in commit a169b30d.

Cantina Managed: Fix verified.

3.3.3 `fulfillRedeem` can burn queued shares for zero assets

Severity: Low Risk

Context: NestVaultCore.sol#L441

Description: `fulfillRedeem` computes `_assets = convertToAssets(_shares)` with floor math. If the exchange rate is below `ONE_SHARE` or `_shares` is very small, `_assets` rounds to zero. The function still calls `_exit` with `_assets = 0` and burns `_shares`, then records `claimableRedeem.assets += 0` and `claimableRedeem.shares += _shares`. The controller ends up with claimable shares but zero assets; `withdraw` then divides by zero and reverts, and `redeem` succeeds only by giving up shares for no payout. An authorized fulfiller can split a queue into dust-sized calls to destroy a user's redeem value.

Relevant code (`NestVaultCore.fulfillRedeem`):

```
function fulfillRedeem(address _controller, uint256 _shares)
    public
    requiresAuth
    nonReentrant
    returns (uint256 _assets)
{
    NestVaultCoreStorage storage $ = _getNestVaultCoreStorage();

    DataTypes.PendingRedeem storage _request = $.pendingRedeem[_controller];

    if (_request.shares == 0 || _shares > _request.shares) {
        revert Errors.ZERO_SHARES();
    }

    _assets = convertToAssets(_shares); // floor rounding

    _request.shares -= _shares;
    $.totalPendingShares -= _shares;
```

```
    uint256 initialBalance = ERC20(asset()).balanceOf(address(this));
    _exit(address(this), ERC20(asset()), _assets, address(this), _shares);
    uint256 finalBalance = ERC20(asset()).balanceOf(address(this));
    uint256 actualTransfer = finalBalance - initialBalance;

    if (actualTransfer < _assets) {
        revert Errors.TRANSFER_INSUFFICIENT();
    }

    $.claimableRedeem[_controller] = DataTypes.ClaimableRedeem(
        $.claimableRedeem[_controller].assets + actualTransfer,
        $.claimableRedeem[_controller].shares + _shares
    );
    emit RedeemFulfilled(_controller, _shares, _assets, actualTransfer);
}
```

Recommendation: Revert `fulfillRedeem` when `convertToAssets` returns zero or enforce a minimum asset out per fulfillment so queued shares cannot be burned without delivering assets.

Plume Network: Fixed in commit a4874305.

Cantina Managed: Fix verified.

3.3.4 `NestVaultOFT` lacks `share()` function required by EIP-7575

Severity: Low Risk

Context: NestVaultOFT.sol#L29

Description: ERC-7575 requires every vault entry point to expose a non-reverting `share()` view that returns the ERC-20 share token address. `NestVaultOFT` inherits `supportsInterface` from `NestVaultCore`, which advertises the ERC-7575 interface ID, but `NestVaultOFT` does not implement `share()`. Any attempt to call `share()` on this vault will hit the fallback and revert. This violates the spec and breaks integrators that rely on ERC-165 detection to obtain the share token address for approvals, accounting and cross-vault coordination.

Recommendation: Add a compliant `share()` view to NestVaultOFT (e.g., `return address(SHARE);`) and ensure it does not revert, or stop advertising ERC-7575 support if the method is not provided.

Plume Network: Fixed in commit 92bfbcfd.

Cantina Managed: Fix verified.

3.3.5 RedeemRequest event misnamed final field

Severity: Low Risk

Context: IERC7540.sol#L93-L96

Description: The emitted `RedeemRequest` event names its final parameter `assets`, but per ERC-7540 the final field must be `shares` and represent the share amount locked for redemption. Current interface/emit:

```
event RedeemRequest(address indexed controller, address indexed owner, uint256 indexed
↪  requestId, address sender, uint256 assets);
```

Spec definition requires the final input to be named `shares` (uint256). Although the types/indexing match, the name diverges from the standard and is a spec non-compliance that can mislead integrators and indexers expecting the canonical field.

Recommendation: Rename the final event parameter to `shares` to align with ERC-7540.

Plume Network: Fixed in commit ddf147e9.

Cantina Managed: Fix verified.

3.3.6 Deposit event omitted in `NestVaultCore._deposit`

Severity: Low Risk

Context: NestVaultCore.sol#L636-L647

Description: ERC-4626 (and ERC-7575) require that `deposit` and `mint` emit the `Deposit` event. In the OpenZeppelin flow, `_deposit` emits this event alongside minting shares. The override of `_deposit` in `NestVaultCore` replaces the parent logic with an auth check and a call to `_enter`, but never emits `Deposit`. As a result, every `deposit/mint` call on vaults inheriting this core fails to produce the mandated event, breaking indexers and integrations that rely on the standard signal to track deposits and reconcile assets to shares.

Recommendation: Emit the standard `Deposit` event inside the overridden `_deposit` after completing the share minting flow (matching the OZ argument order: `sender`, `owner`, `assets`, `shares`), or invoke the parent `_deposit` while preserving the custom share-handling, so the event is always fired.

Plume Network: Fixed in commit 63cd6f4d.

Cantina Managed: Fix verified.

3.3.7 EIP-712 domain can drift on `authorizeOperator`

Severity: Low Risk

Context: NestVaultCore.sol#L340

Description: The function `authorizeOperator` from contract `NestVaultCore` verifies EIP-712 signatures using the domain initialized in `__NestVaultCore_init`. This domain is built via `EIP712Upgradeable` using `SHARE.name()` at initialization time. At the same time, the `NestShare` contract allows changing the share name and symbol after deployment through `setNameAndSymbol`.

Because the EIP-712 domain separator is computed and cached using the original `SHARE.name()`, a subsequent call to `NestShare.setNameAndSymbol` creates a mismatch between:

- The off-chain domain data that signers will typically use (the current `name()`), and.
- The on-chain EIP-712 domain used by `authorizeOperator`, which still embeds the old name.

After a rename, signatures produced off-chain using the updated name are hashed under a different EIP-712 domain than the one used by `authorizeOperator`. These signatures are rejected with `INVALID_SIGNER` even though they correspond to the intended operator authorization. In practice, once the share name (or symbol) is updated, new `authorizeOperator` signatures using the updated metadata cannot be used, and operator authorization via EIP-712 becomes unreliable unless signers keep using the original name.

Recommendation: Consider to make the EIP-712 domain independent from mutable token metadata, for example by:

- Preventing post-initialization changes to the share name and symbol, or.
- Using a separate, immutable domain name for `EIP712Upgradeable` (for example, a fixed string unrelated to `SHARE.name()`), or.
- Explicitly bumping and recomputing the EIP-712 domain whenever the share metadata changes, and documenting the domain versioning for integrators.

This would consider to keep the domain used off-chain for signing and the domain used on-chain in `authorizeOperator` aligned over time.

Plume Network: Fixed in commit a203e928.

Cantina Managed: Fix verified.

3.4 Gas Optimization

3.4.1 Cache asset token in redeem paths

Severity: Gas Optimization

Context: NestVaultCore.sol#L497-L501

Description: Both `fulfillRedeem` and `instantRedeem` call `asset()` multiple times to build an `ERC20` wrapper and fetch balances. Each call reloads the same storage slot and repeats the wrapper construction. Caching the asset token once per call avoids redundant SLOADs and bytecode in these paths, saving gas whenever redemptions are executed.

Recommendation: Inside `fulfillRedeem` and `instantRedeem`, store `ERC20 assetToken = ERC20(asset());` once and reuse it for balance checks and `_exit` calls to eliminate duplicate storage reads.

Plume Network: Fixed in commit 4e766785.

Cantina Managed: Fix verified.

3.4.2 RateProvider lookup on base-asset fee claims

Severity: Gas Optimization

Context: NestAccountant.sol#L202-L204

Description: `claimFees` loads `rateProviderData` for the fee asset before checking whether the fee asset is the base token. When claiming in base, the provider data is unused, but the storage read still happens. This adds an extra SLOAD and bytecode for the common base-asset path without changing behaviour.

Recommendation: Branch on `if (_feeAsset == base)` first and return the owed amount directly; only load `rateProviderData` and perform conversions in the non-base branch to avoid the unnecessary storage read.

Plume Network: Fixed in commit c1b0f2d2.

Cantina Managed: Fix verified.

3.4.3 Avoid double storage read in `getRateSafe`

Severity: Gas Optimization

Context: NestAccountant.sol#L379-L387

Description: `getRateSafe` loads the accountant storage to check `isPaused` and then calls `getRate()`, which reloads the storage pointer and reads `exchangeRate` again. This adds an extra SLOAD and call frame for a simple getter. Returning `$.accountantState.exchangeRate` directly after the pause check avoids the redundant read and bytecode.

Recommendation: After the pause check, return the exchange rate from the same storage pointer instead of calling `getRate()`, eliminating the extra storage access.

Plume Network: Fixed in commit 231b99e1.

Cantina Managed: Fix verified.

3.5 Informational

3.5.1 Cross-chain shares minted without moving backing

Severity: Informational

Context: NestShare.sol#L25

Description: `NestShare` bridges by burning on the source chain and minting the same amount on the destination, but no underlying assets move with those shares. The LayerZero `_debit/_credit` flow only burns/mints; `NestShare.enter/exit` transfers assets only when `assetAmount > 0`, so cross-chain minting is unbacked. Each chain's redemptions pay out from that chain's local `NestShare` assets, there is no built-in cross-chain settlement or solvency check.

Example:

- Chain A: 1,000 assets backing 1,000 shares.
- Chain B: 5,000 assets backing 5,000 shares.

- Bridge 1,000 shares from B → A (no assets move).
- Result: A has 1,000 assets and 2,000 shares; B has 5,000 assets and 4,000 shares. A is undercollateralized; B is overcollateralized. Redeemers on A may be paid less or revert once A's assets are exhausted.

Because exchange-rate math is global while solvency is per-chain, share face value can diverge from actual redeemable value on underfunded chains. Users can mint where assets exist, bridge shares to another chain and redeem against that chain's pool, draining it without surrendering the originating backing.

Recommendation: Periodically bridge/move underlying assets between chains to match the shares currently present on each chain, so per-chain backing aligns with outstanding shares. If periodic rebalancing is the chosen model, add monitoring/alerts and guards to pause or limit redemptions/bridging when local backing falls short.

Plume Network: Acknowledged. This behavior is intentional. `NestShare` is a cross-chain receipt token: bridging only moves shares, while the underlying moves asynchronously during vault management and redemption fulfillment. Solvency is enforced globally, not per-chain at every moment. If a chain becomes temporarily over/undercollateralized after a bridge, the imbalance is corrected during periodic rebalancing or when fulfilling redemptions (which includes moving underlying assets from chains with surplus). The scenario described is expected under the async model and is mitigated by global solvency, monitoring, and operational rebalancing.

Cantina Managed: Acknowledged.

3.5.2 `updateRedeem` cannot increase pending shares

Severity: Informational

Context: NestVaultCore.sol#L514

Description: `NestVaultCore.updateRedeem` is meant to handle both reductions and increases of a pending redeem, but the pre-check `if (_oldShares < _newShares) revert Errors.INSUFFICIENT_BALANCE();` makes the "increase" path unreachable. Any attempt to raise `_newShares` always reverts before the branch that would pull additional shares, contrary to the docstring comment that says increases are allowed. Because of this, pending redeems can only be decreased or left unchanged and legitimate increases can not be made.

```
function updateRedeem(uint256 _newShares, address _controller, address _owner) external
↪   requiresAuth {
    NestVaultCoreStorage storage $ = _getNestVaultCoreStorage();

    if (_owner != msg.sender && !$.isOperator[_owner][msg.sender]) {
        revert Errors.UNAUTHORIZED();
    }
    uint256 _oldShares = $.pendingRedeem[_controller].shares;

    if (_oldShares == 0) {
        revert Errors.NO_PENDING_REDEEM();
    }

    if (_oldShares < _newShares) {
        revert Errors.INSUFFICIENT_BALANCE();
    }

    if (_newShares == _oldShares) {
        return;
    }

    uint256 _returnAmount = _oldShares - _newShares;

    $.totalPendingShares -= _returnAmount;

    $.pendingRedeem[_controller].shares = _newShares;

    SafeTransferLib.safeTransfer(ERC20(address(SHARE)), _owner, _returnAmount);
```

```
    emit RedeemUpdated(_controller, _owner, msg.sender, _oldShares, _newShares);
}
```

Recommendation: Allow the increase branch by moving the balance check into that branch and removing the unconditional revert by, for example, comparing balances only when `_newShares > _oldShares`, then transfer the delta in and update totals.

Plume Network: Fixed in commit 1004c716.

Cantina Managed: Fix verified.

3.5.3 Unused code

Severity: Informational

Context: NestVault.sol#L7-L10

Description: Several files pull in libraries or interfaces that are never referenced: `NestVaultPredicateProxy` imports `FixedPointMathLib` and declares `using FixedPointMathLib for uint256` without calling its functions. `NestVault` imports `IERC20` and `SafeERC20` but uses neither. `NestVaultOFT` imports `IERC20` but relies on `NestShare`/solmate types instead. Keeping unused imports and using-directives increases bytecode size and lint noise without any functional benefit.

Recommendation: Remove the unused imports/using directives from these contracts.

Plume Network: Fixed in commit ea804044.

Cantina Managed: Fix verified.

3.5.4 Predicate approvals not bound to function/args

Severity: Informational

Context: NestVaultPredicateProxy.sol#L84-L87

Description: Both `deposit` and `mint` build the predicate payload as `abi.encodeWithSignature("deposit()")`, so the signed task never commits to the actual function selector or arguments. A single predicate signature can be replayed by the same caller across both code paths and with arbitrary `_depositAsset`, `_vault`, `_recipient` and amounts until expiry. Off-chain policy cannot enforce per-function or per-parameter limits. Approving a "deposit" authorizes unlimited deposits or mints by that caller within the validity window. Because there is no on-chain nonce/use tracking, the same predicate can also be reused multiple times.

Recommendation: Document this behaviour. If it is not intended, consider including the exact function selector and hashed arguments in the predicate payload (separate payloads for `deposit` and `mint`) and add a nonce or single-use marker so signatures can not be replayed across functions or parameter changes.

Plume Network: Fixed in commit c0655d42.

Cantina Managed: Fix verified.

3.5.5 `SupportsInterface` NatSpec mismatches implementation

Severity: Informational

Context: NestShare.sol#L189-L196

Description: The `NestShare.supportsInterface` NatSpec says the contract supports `ERC7540Operator` and `ERC7540Redeem`, but the implementation actually returns support for `IERC7575Share`, `IOFT`, `IERC20` and `IERC165`. The code is correct however, the comment is stale and can mislead integrators about which interfaces are advertised.

Recommendation: Update the NatSpec to reflect the actual supported interfaces, or adjust the return logic if the intent was to match the documented interfaces.

Plume Network: Fixed in commit 914c9d82.

Cantina Managed: Fix verified.

3.5.6 Initialize docstring misstates LayerZero delegate

Severity: Informational

Context: NestVaultOFT.sol#L39

Description: `NestVaultOFT.initialize`'s comment says it "Initializes _owner as the LayerZero delegate," but the code passes `_delegate` to `__OFTCore_init` and `_owner` to `Auth`. The LayerZero delegate is `_delegate`, not `_owner`. The docstring is misleading relative to the implementation.

Recommendation: Update the docstring to state that `_delegate` is set as the LayerZero delegate and `_owner` is used for Auth, so documentation matches the code.

Plume Network: Fixed in commit aac4a2cb.

Cantina Managed: Fix verified.

3.5.7 Auth gating conflicts with ERC7540 public entrypoints

Severity: Informational

Context: NestVaultCore.sol#L381

Description: Several ERC7540/7575 user-facing entrypoints are gated by `requiresAuth` or `isAuthorized`, meaning only callers whitelisted in the Authority can use them. EIP-7540 requires controllers (or approved operators) to be able to call these functions directly. A restrictive authority configuration would block normal users and break spec compliance. Affected functions include `requestRedeem` (async redeem queue), `withdraw`, `redeem`, `instantRedeem`, and the internal `_deposit/_enter` path used by deposits and mints (which enforces `isAuthorized`). Without explicit public capabilities configured, controllers with ERC20 allowances cannot initiate these flows.

Recommendation: Configure the Authority to mark these entrypoints as public capabilities for the vault address (e.g., via `setPublicCapability` for `requestRedeem`, `withdraw`, `redeem`, `instantRedeem`, and deposit/mint), or remove the auth gates for user-facing ERC7540/ERC4626 functions. Document this as a required deployment invariant to maintain ERC7540/7575 compliance.

Plume Network: Acknowledged. The entry functions (`deposit` and `mint`) are gated to honour compliance checks for AML/KYC via third party provider called Predicate. However, we'll consider setting public capabilities for `requestRedeem`, `withdraw`, `redeem` and `instantRedeem` subject to business requirements of compliance flow.

Cantina Managed: Acknowledged.

3.5.8 Avoid usage of magic numbers

Severity: Informational

Context: NestVaultCore.sol#L210

Description: The function `__NestVaultCore_init` from contract `NestVaultCore` sets the maximum instant redemption fee using a hard-coded literal value (`200_000`) assigned to `maxFees[DataTypes.Fees.InstantRedemption]`. Similar literals are used in other parts of the codebase.

Using magic numbers makes it harder to understand what the value represents (for example, whether it encodes 20%, a basis-points value, or a fixed scaling factor), and increases the chance of configuration or maintenance mistakes if the same value needs to be updated in multiple places. It also makes external review and auditing less straightforward, because the semantics of the number are not explicit at the declaration site.

Recommendation: Consider to replace magic numbers with named constants or clearly documented configuration parameters. For example, consider to define a dedicated constant for the instant redemption fee cap and reuse it in initialization and checks. Applying the same pattern to other fee or scaling values can improve readability and reduce the risk of misconfiguration when the system evolves.

Plume Network: Fixed in commit 1511ca62.

Cantina Managed: Fix verified.